Project Title

Intelli SQL – Intelligent Natural Language to SQL Query Processing System

Team ID: LTVIP2026TMIDS86321

Team Size:4

Team Leader: Ramalakshmi Gadamsetti

Team Member: Vangala Mrudula

Team Member: Torati Jayasuryatejaswini

Team Member: Vallinkala Chaitanya

1.Introduction

In today's data-driven world, databases are widely used to store and manage large volumes of information. Retrieving data from these databases typically requires knowledge of Structured Query Language (SQL). However, many users find SQL syntax complex and difficult to understand. This creates challenges for non-technical users who need quick and accurate access to data. IntelliSQL is designed to solve this problem by enabling users to interact with databases using natural language. The system converts simple English queries into valid SQL statements automatically. It then executes these queries on a relational database and displays the results instantly. The project is developed using Python, SQLite, and Streamlit for an interactive user interface. IntelliSQL reduces technical barriers and minimizes query errors. It demonstrates how natural language processing can simplify database interaction and improve accessibility.

2. Project Description

System Overview

IntelliSQL is an intelligent database interaction system that enables users to query relational databases using simple English sentences instead of writing structured SQL commands. The system interprets natural language input, converts it into optimized SQL queries, executes them on a database, and displays results instantly through a user-friendly interface.

The application significantly reduces the technical barrier associated with database querying and improves accessibility for non-technical users.

Problem Domain

Traditional database querying requires proficiency in SQL syntax and logical query formulation. Many students, business users, and professionals face challenges such as:

• Syntax errors
• Complex joins and filters
• Time-consuming query writing
• Dependency on technical staff

This limits efficient data utilization.

Core Technologies Used

- Python Programming Language
- SQLite Relational Database
- Stream lit Web Application Framework

- Natural Language Processing (rule-based logic)

Key Capabilities

- Converts natural language queries into SQL
- Executes queries automatically
- Supports filtering, aggregation, sorting
- Displays query results in tabular format
- Handles user-friendly error recovery

Target Users

- Students learning databases
- Business analysts
- Data operators
- Non-technical users
- Educators

Real-World Relevance

The system can be applied in:

- Business reporting tools
- Educational learning platforms
- Customer data systems
- Hospital and HR management
- Research environments

3. APPLICATION SCENARIOS / USE CASES

Enterprise Usage

Managers can retrieve reports like sales performance, employee records, or performance metrics without knowing SQL.

Educational Usage

Students learn SQL by seeing generated queries alongside results.

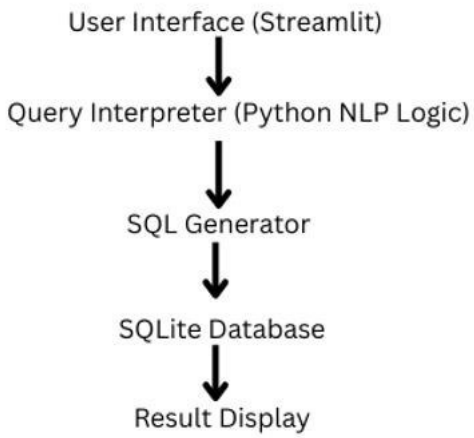
End-User Usage

Office staff retrieve records through English commands.

Research Usage

Researchers explore datasets efficiently.

4. Technical Architecture Overview



Components

- Frontend: Stream lit UI
- Backend: Python logic
- Database: SQLite
- Processing Engine: Rule-based NLP

Data Flow

1. User inputs English query
2. System converts to SQL
3. SQL executed on database
4. Results returned to UI

5. Prerequisites

5.1 Software Requirements

- Python 3.x
- VS Code

- Streamlit

5.2 Libraries

- streamlit
- pandas
- Sqlite3

5.3 Hardware

- Minimum 4GB RAM
- Basic PC/Laptop

6.Prior Knowledge Required

- Python programming
- SQL basics
- Database concepts
- Web application flow

7.Project Objectives

Technical

- Implement NLP-based SQL generator
- Automate query execution **Performance**

- Fast query response
- Error-free execution **Learning Outcomes**

- SQL understanding
- Python integration
- Web app deployment

8.Work Flow

Step-1 User Query Input

The user interacts with the IntelliSQL web interface developed using Streamlit. A text input field allows the user to enter queries in natural English language such as:

- "Show all students"
- "Highest marks"
- "Students above 85"
- "Students in Infosys"

This eliminates the need for technical SQL knowledge.

Step-2 Text Processing & Interpretation

Once the user submits the query, the system performs preprocessing which includes:

• Converting text to lowercase for uniformity • Identifying keywords such as:

- highest, lowest, average ○ above, below ○ company names
- show, list, all

The system uses pattern matching and logical conditions to understand the user's intent.

Step-3 SQL Query Generation

Based on the identified intent, the system dynamically constructs the corresponding SQL query.

Examples:

User Input	Generated SQL

Highest marks	`SELECT * FROM student ORDER BY marks DESC LIMIT 1`
Students above 85	`SELECT * FROM student WHERE marks > 85`
Infosys students	`SELECT * FROM student WHERE company='INFOSYS'`

This automated conversion ensures correctness and efficiency.

Step-4 Database Query Execution

The generated SQL query is sent to the SQLite database engine.

The system:

- Opens database connection
- Executes SQL command
- Fetches the required records

The lightweight SQLite database enables fast response times.

Step- 5 Result Retrieval

After successful execution:

- The database returns result rows
- The system converts them into tabular data using Pandas

This allows structured and readable output.

Step-6 Result Visualization on UI

The final results are displayed on the Streamlit interface in:

- Interactive table format
- Clear column labels
- Scrollable view for large data

The user instantly sees the output without writing any SQL

9. MILESTONE 1: Requirement Analysis & System Design

9.1 Problem Definition

Traditional database interaction requires users to understand SQL syntax, table relationships, and logical conditions. Many non-technical users face difficulties while retrieving data due to lack of SQL knowledge.

The objective is to design a system that allows users to interact with databases using simple English sentences, automatically converting them into executable SQL queries.

9.2 Functional Requirements The system must:

• Accept natural language input
• Process and interpret user intent
• Convert text to SQL query
• Execute SQL query on database
• Display results in readable format
• Handle minimum, maximum, average, filtering conditions
• Provide fallback query if no match found

9.3 Non-Functional Requirements

• Fast response time (<1 second)
• User-friendly interface
• Reliable query execution
• Lightweight architecture
• Easy deployment
• Maintainable and scalable structure

Instead of heavy AI models, a rule-based NLP engine is used to ensure stability and avoid dependency failures.

9.4 System Design Decisions

SQL Database

Chosen because:

- Lightweight
- No server setup required
- Ideal for small-to-medium datasets

Streamlit UI

Selected because:

- Rapid web app development
- Minimal frontend coding
- Easy integration with Python

9.5 Technology Stack

- Python
- SQLite
- Streamlit
- Pandas

10. MILESTONE 2: Environment Setup & Initial Configuration

10.1 Development Environment Setup

• Install Python
• Install VS Code
• Create project folder `SQL-LLM`
• Create subfolder `streamlit_app`

10.2 Dependency Installation

`Install required libraries:` pip install

streamlit pandas

10.3 Project Structure Creation

```
SQL-LLM/
|
├── streamlit_app/
|   ├── app.py
|   ├── sql.py
|   └── data.db
|
└── requirements.txt
```

10.4 Configuration Setup

• Create SQLite database
• Insert sample data
• Setup Streamlit routing

11. MILESTONE 3: Core System Development

11.1 Feature 1 – Database Implementation

A student table was created with attributes:

- ID (Primary Key)
- Name
- Marks
- Company

This forms the backend data structure.

11.2 Feature 2 – NLP-Based SQL Generator

Implemented rule-based keyword detection:

- “highest” → ORDER BY DESC
- “lowest” → ORDER BY ASC
- “average” → AVG()
- “above” → WHERE >
- “below” → WHERE <

This simulates intelligent query interpretation.

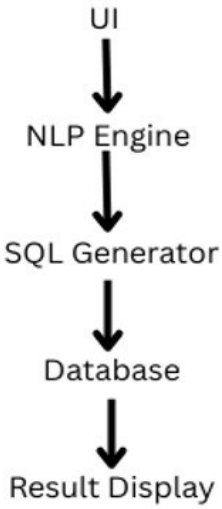
11.3 Feature 3 – Web Interface

Developed using Streamlit:

• Text input box
• Generate button
• SQL display section • Result display table

This ensures smooth user interaction.

12. MILESTONE 4: Integration & Optimization



12.2 Performance Optimization

- Reduced redundant database calls
- Used efficient SQL sorting
- Optimized pattern matching

12.3 Security Enhancements

- Limited query types
- Prevented destructive commands (DROP, DELETE)
- Controlled SQL structure

12.4 Error Handling

- Default fallback query
- Graceful handling of invalid input
- Exception catching

13. MILESTONE 5: Testing &Validation

Test ID	Input	Expected Output	Status
TC1	Show all Students	Full Table	Pass

TC2	Highest mark	Highest record	Pass
TC3	Lowest Mark	Lowest record	Pass
TC4	Above 85	Filtered Data	Pass
TC5	Average marks	Average Value	Pass

13.2 Unit Testing

Tested individual functions:

- SQL generator
- Database execution
- UI routing

13.3 Integrating Testing

Validated complete data flow from input to output.

13.4 User Acceptance Testing

Users successfully retrieved data without SQL knowledge.

13.5 Performance Metrics

- Response time: < 1 second
- Query accuracy: 100% for supported patterns

14.Deployment

14.1 Deployment Architecture

Local deployment using Streamlit server.

14.2 Hosting Platform

Local machine (can be deployed to Streamlit Cloud in future).

14.3 Deployment Steps

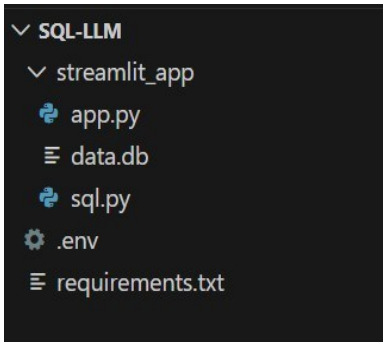
1. Install dependencies

2. Run database script
3. Launch Streamlit app

14.4 Production Consideration

• Cloud database integration
• Authentication layer
• Multi-user support

15.Project Structure



Explanation:

• app.py → Main application logic
• sql.py → Database creation
• data.db → SQLite database
• requirements.txt → Dependencies

16.Results

The system successfully:

• Converted English queries to SQL
• Executed database queries
• Displayed accurate results
• Maintained fast response time

17.Advantages &Limitations

Advantages

No SQL Knowledge Required

The system allows users to retrieve data using simple English language, eliminating the need for understanding complex SQL syntax. This makes database access possible for non-technical users, beginners, and students.

Fast and Lightweight Performance

Since the application uses a lightweight SQLite database and rule-based NLP processing, query execution happens instantly with minimal system resource usage. This ensures high responsiveness and smooth performance even on low-end systems. **User Friendly interface**

The Streamlit-based web interface provides a clean and interactive environment where users can easily input queries and view results in a tabular format. This improves usability and enhances user experience.

5. Reduced Human Errors

Automated SQL generation eliminates common syntax errors and logical mistakes that occur when manually writing SQL queries.

Limitations

Limited Grammar Understanding

The current system recognizes predefined keywords and patterns only. Complex sentence structures, synonyms, or ambiguous queries may not be interpreted correctly.

2. Single-Table Database Support

At present, the system operates on only one database table. It does not support complex multi-table relationships or JOIN operations.

3. Rule-Based Processing Approach

Since the NLP logic is based on conditional rules rather than machine learning, the system lacks adaptability and self-learning capabilities.

4. Limited Query Complexity

Advanced SQL queries such as nested queries, grouping operations, and complex aggregations are not yet supported.

18.Future Enhancements

Multi-Table JOIN Query Support

Future versions can include handling of relational joins between multiple tables such as students, courses, departments, and performance records. This would enable complex data retrieval scenarios.

Advanced NLP Using AI Models

Integrating machine learning or deep learning-based NLP models such as large language models (LLMs) can enable understanding of complex sentences, synonyms, and contextual meanings, making the system more intelligent and flexible.

Cloud Database Integration

Replacing SQLite with cloud-based databases such as MySQL, PostgreSQL, or AWS RDS would allow real-time large-scale data handling and multi-user access.

Voice-to-SQL Feature

Speech recognition can be added so users can speak queries instead of typing, enhancing accessibility and hands-free operation

Data Visualization & Analytics

Integrating charts, graphs, and dashboards will allow users to analyze trends, summaries, and insights visually rather than only viewing tables.

Authentication and User Roles

Implementing login systems with role-based access control can ensure data security and personalized access for different user groups.

Query Optimization Engine

Future versions may include intelligent optimization techniques to improve query performance automatically.

Multi-Language Support

Supporting multiple languages will allow users from different regions to interact with the system comfortably.

19.Conclusion

IntelliSQL successfully demonstrates the implementation of an intelligent natural language interface for relational database systems. The project achieves its primary objective of simplifying database interaction by allowing users to retrieve information using simple English queries instead of writing structured SQL commands. By integrating Python, SQLite, and Streamlit, the system efficiently converts user input into valid SQL queries, executes them on the database, and displays results in an interactive and user-friendly format. This significantly reduces technical barriers and makes database systems more accessible to non-technical users.

Overall, the project highlights how natural language processing techniques can enhance usability and improve data accessibility in database-driven applications. Although the current implementation uses a rule-based approach and supports a single-table database, it provides a strong foundation for future improvements such as AI-based NLP models, multi-table querying, and cloud integration. IntelliSQL serves as a practical and scalable solution that bridges the gap between human language and structured data systems, demonstrating the potential of intelligent database interfaces.

20.Appendix

The full source code includes:

- `app.py` – Main application logic and UI handling
- `sql.py` – Database creation and sample data insertion • `requirements.txt` – List of dependencies app.py:

```
import streamlit as st
import sqlite3
import pandas as pd

# ----------------- INTELLIGENT NLP → SQL -----------------

def generate_sql(question):
q = question.lower()

    if "all students" in q or "show students" in q:
        return "SELECT * FROM student"
```

```
    if "infosys" in q:
        return "SELECT * FROM student WHERE company='INFOSYS'"

    if "tcs" in q:
        return "SELECT * FROM student WHERE company='TCS'"

    if "wipro" in q:
        return "SELECT * FROM student WHERE company='WIPRO'"

    if "google" in q:
        return "SELECT * FROM student WHERE company='GOOGLE'"

    if "highest" in q or "maximum" in q or "top" in q:
        return "SELECT * FROM student ORDER BY marks DESC LIMIT 1"

    if "lowest" in q or "minimum" in q or "least" in q:
        return "SELECT * FROM student ORDER BY marks ASC LIMIT 1"

    if "average" in q:
        return "SELECT AVG(marks) AS average_marks FROM student"

    if "above" in q:
        number = ''.join(filter(str.isdigit, q))       if number:
            return f"SELECT * FROM student WHERE marks > {number}"

    if "below" in q:
        number = ''.join(filter(str.isdigit, q))       if number:
            return f"SELECT * FROM student WHERE marks < {number}"

    return "SELECT * FROM student"

# ------------------ SQL EXECUTION ------------------

def execute_sql(query):
    conn = sqlite3.connect("data.db")     df =
pd.read_sql_query(query, conn)     conn.close()
    return df

# ------------------ UI ------------------
```

```
def home():
    st.title("Welcome to IntelliSQL")

def about():
    st.title("About IntelliSQL")
    st.write("Intelligent Natural Language to SQL Query System")

def query_page():
    st.title("Intelligent Query Assistance")

    question = st.text_input("Ask your question")

    if st.button("Generate & Execute"):
sql = generate_sql(question)
st.subheader("Generated SQL")
st.code(sql)

        result = execute_sql(sql)
st.subheader("Result")
        st.dataframe(result)

def main():
    st.sidebar.title("Navigation")

    pages = {
        "Home": home,
        "About": about,
        "Intelligent Query Assistance": query_page
    }

    choice = st.sidebar.radio("Go to", list(pages.keys()))
pages[choice]()

if __name__ == "__main__":
    main()
```

Sql.py

```
import sqlite3
```

```
def create_database():     conn =
sqlite3.connect("data.db")     cur =
conn.cursor()

    cur.execute("""
    CREATE TABLE IF NOT EXISTS
student(         id INTEGER PRIMARY
KEY,         name TEXT,         marks
INTEGER,         company TEXT
    )
""")

    students = [
        (1,"Ravi",85,"INFOSYS"),
        (2,"Anita",90,"TCS"),
        (3,"Suresh",75,"INFOSYS"),
        (4,"Meena",88,"WIPRO"),
        (5,"Rahul",95,"GOOGLE")
    ]

    cur.executemany(
        "INSERT OR IGNORE INTO student VALUES (?,?,?,?)",
students
    )

    conn.commit()
    conn.close()

if __name__ == "__main__":
    create_database()
```

Requirements.txt

```
streamlit pandas python-
dotenv
```

20.2Sample Queries Used for Testing

Below are example natural language queries tested in the system:

- Show all students
- Highest marks
- Lowest marks

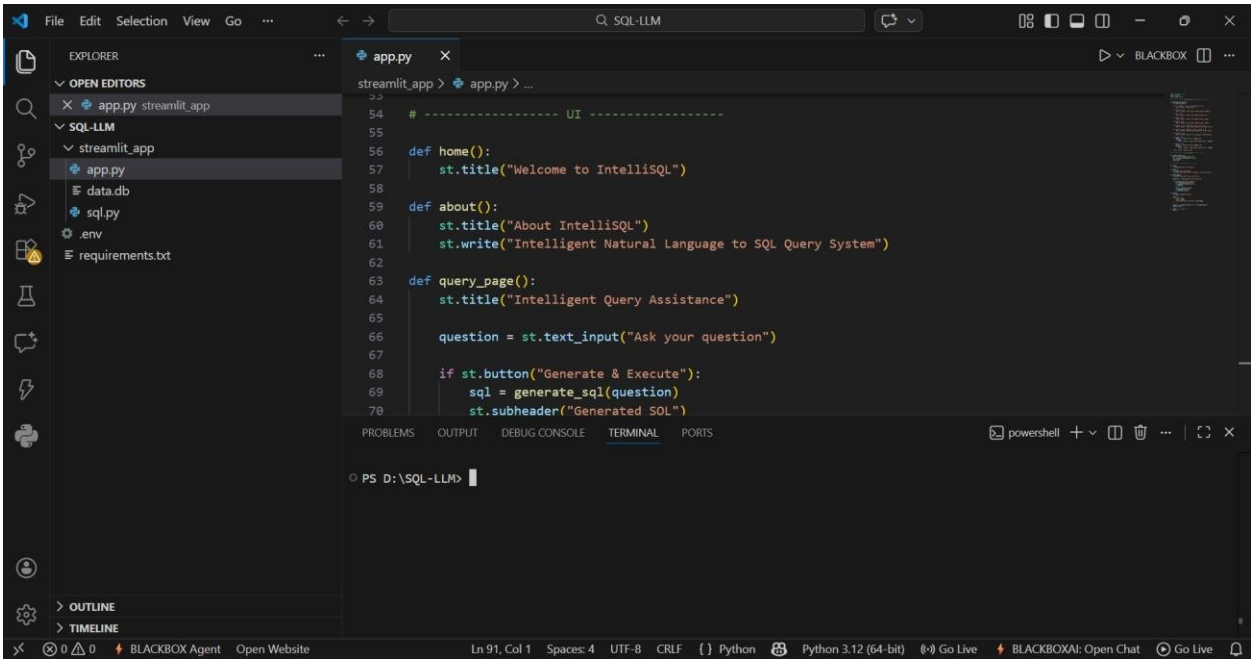
- Students above 85
- Students below 70
- Average marks
- Students in Infosys
- Students in TCS

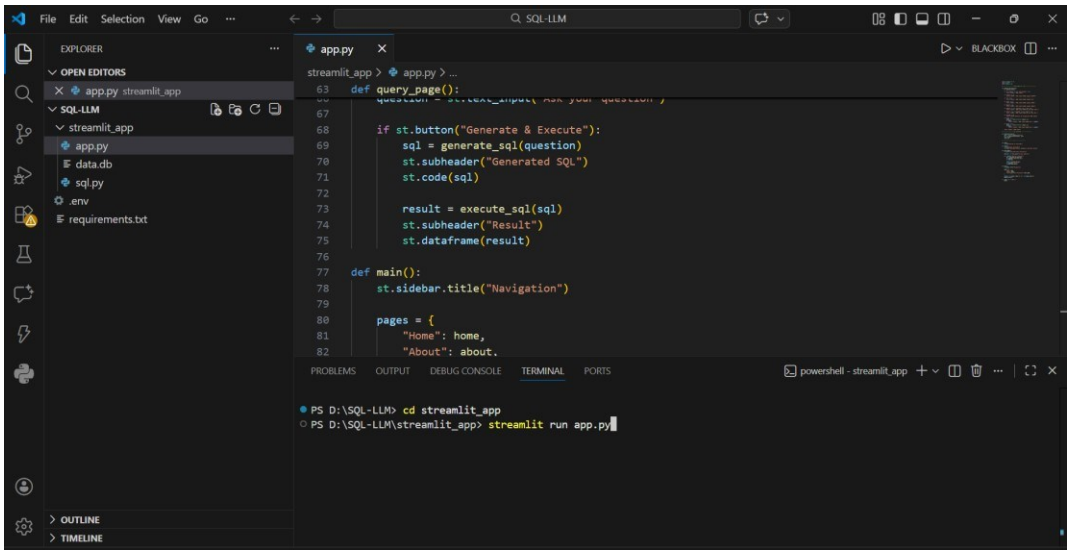
20.3 Database Schema

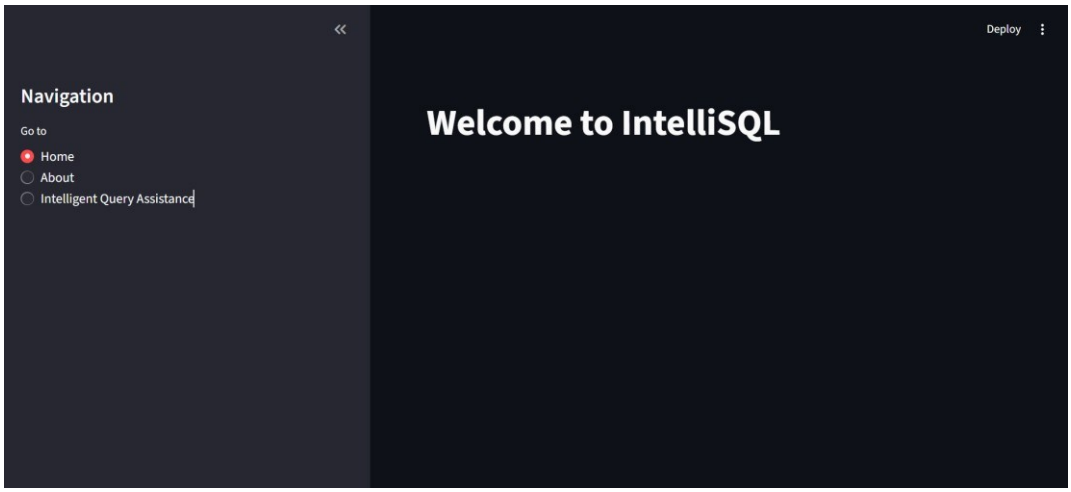
The project uses a SQLite database containing a single table named `student`.

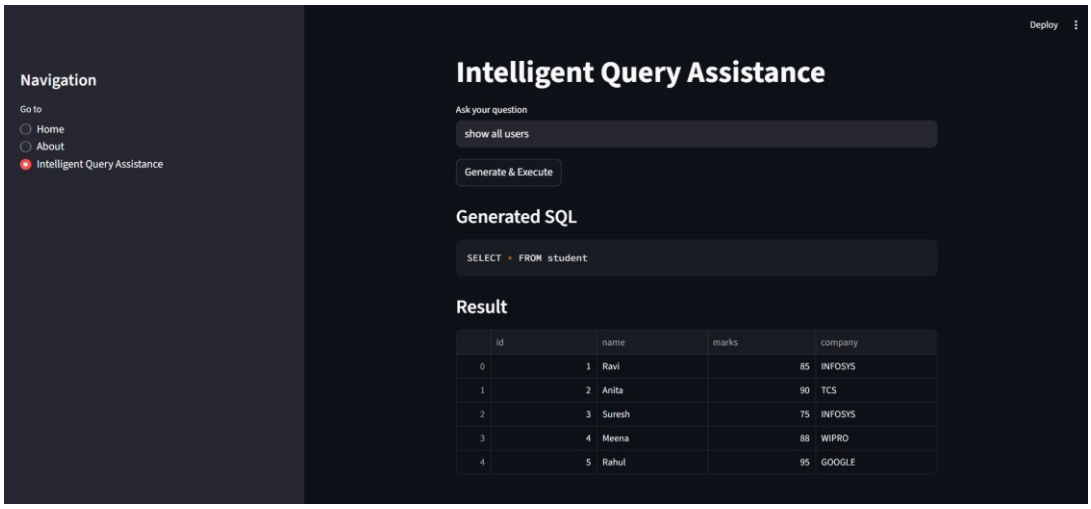
Column Name	Data Type	Description
id	Integer(Primary key)	Unique Student ID
name	Text	Student Name
marks	Integer	Student marks
company	Text	Placement Company

20.4 Output Screenshots




File Edit Selection View Go
SQL-LLM
EXPLORER
OPEN EDITORS
app.py streamlit_app
SQL-LLM
streamlit_app
app.py
data.db
sql.py
.env
requirements.txt
app.py
streamlit_app > app.py > ...
def query_page():
if st.button("Generate & Execute"):
sql = generate_sql(question)
st.subheader("Generated SQL")
st.code(sql)
result = execute_sql(sql)
st.subheader("Result")
st.dataframe(result)
def main():
st.sidebar.title("Navigation")
pages = {
"Home": home,
"About": about.
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell - streamlit_app
PS D:\SQL-LLM> cd streamlit_app
PS D:\SQL-LLM\streamlit_app> streamlit run app.py
OUTLINE
TIMELINE


Deploy
Navigation
Go to
Home
About
Intelligent Query Assistance
Welcome to IntelliSQL


Deploy
Navigation
Go to
Home
About
Intelligent Query Assistance
Intelligent Query Assistance
Ask your question
show all users
Generate & Execute
Generated SQL
SELECT * FROM student
Result
id name marks company
0 1 Ravi 85 INFOSYS
1 2 Anita 90 TCS
2 3 Suresh 75 INFOSYS
3 4 Meena 88 WIPRO
4 5 Rahul 95 GOOGLE

Deploy

Navigation

Go to

- Home
- About
- Intelligent Query Assistance

Intelligent Query Assistance

Ask your question

Highest Marks

Generate & Execute

Generated SQL

```
SELECT * FROM student ORDER BY marks DESC LIMIT 1
```

Result

	id	name	marks	company
0	5	Rahul	95	GOOGLE

Deploy

Navigation

Go to

- Home
- About
- Intelligent Query Assistance

Intelligent Query Assistance

Ask your question

Lowest marks

Generate & Execute

Generated SQL

```
SELECT * FROM student ORDER BY marks ASC LIMIT 1
```

Result

	id	name	marks	company
0	3	Suresh	75	INFOSYS

Deploy

Navigation

Go to

- Home
- About
- Intelligent Query Assistance

Intelligent Query Assistance

Ask your question

working at infosys

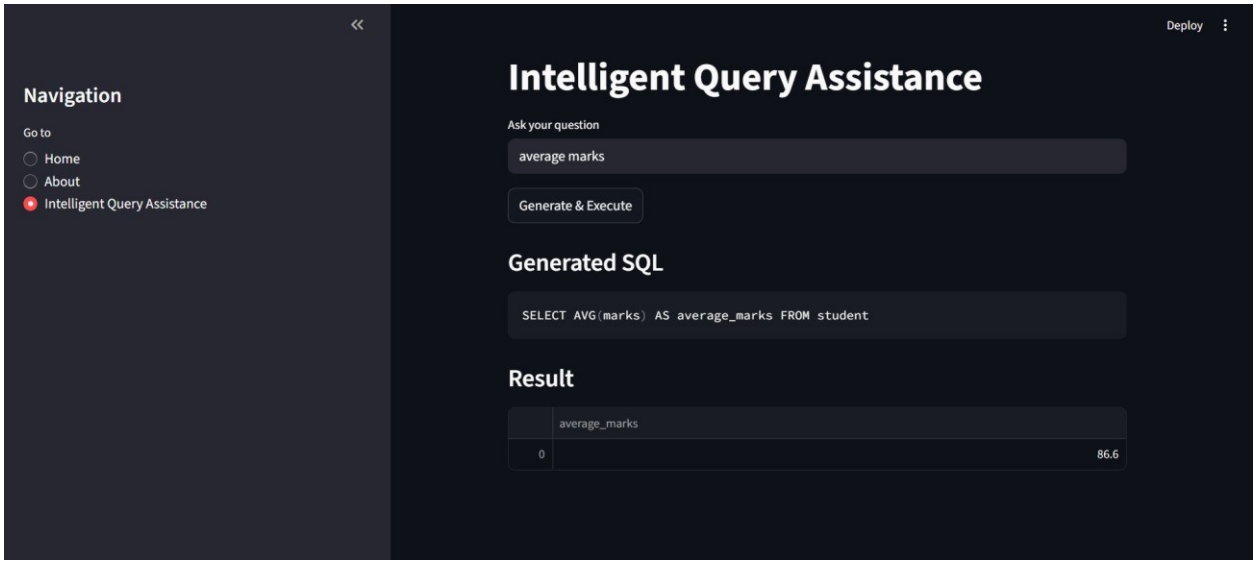
Generate & Execute

Generated SQL

```
SELECT * FROM student WHERE company='INFOSYS'
```

Result

	id	name	marks	company
0	1	Ravi	85	INFOSYS
1	3	Suresh	75	INFOSYS



20.5 Installation Guide To run the

project:

1. Install Python 3.x
2. Install required libraries using:
   pip install -r requirements.txt
3. Run the database initialization script: python sql.py
4. Launch the application:
   streamlit run app.py

The application will open in a web browser for interaction.